

2026.04.20

SPI & I2C 설계 및 UVM 검증

김수빈

Contents

01

개요

02

SPI 설계 및 UVM

03

I2C 설계 및 UVM

04

동작 영상

05

Troubleshooting

개요

| Project 목적

01	SPI/I2C 통신 프로토콜 동작 원리 분석
02	통신 프로토콜 기반 RTL 설계
03	UVM 기반 검증 환경 구축 및 기능 검증

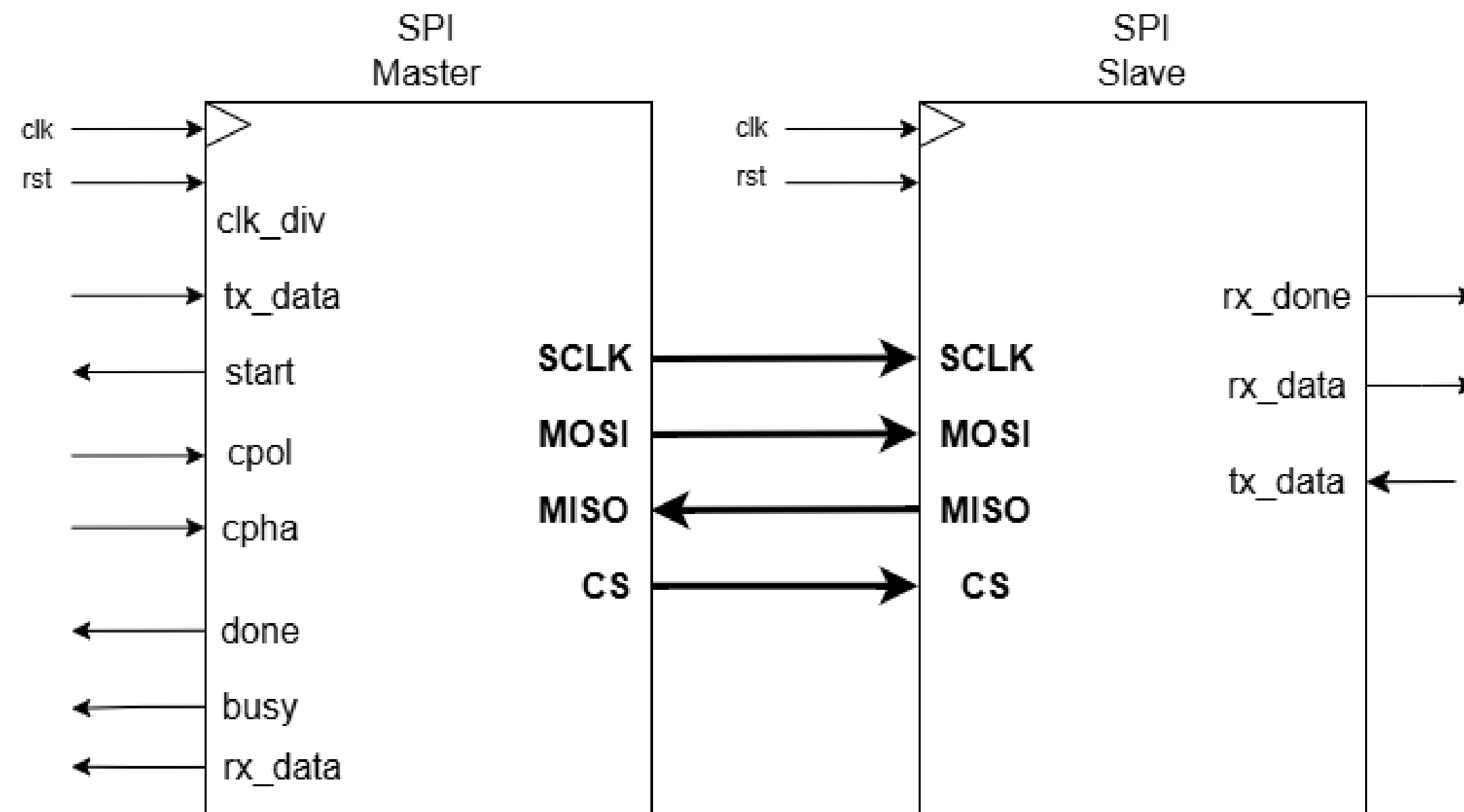
SPI



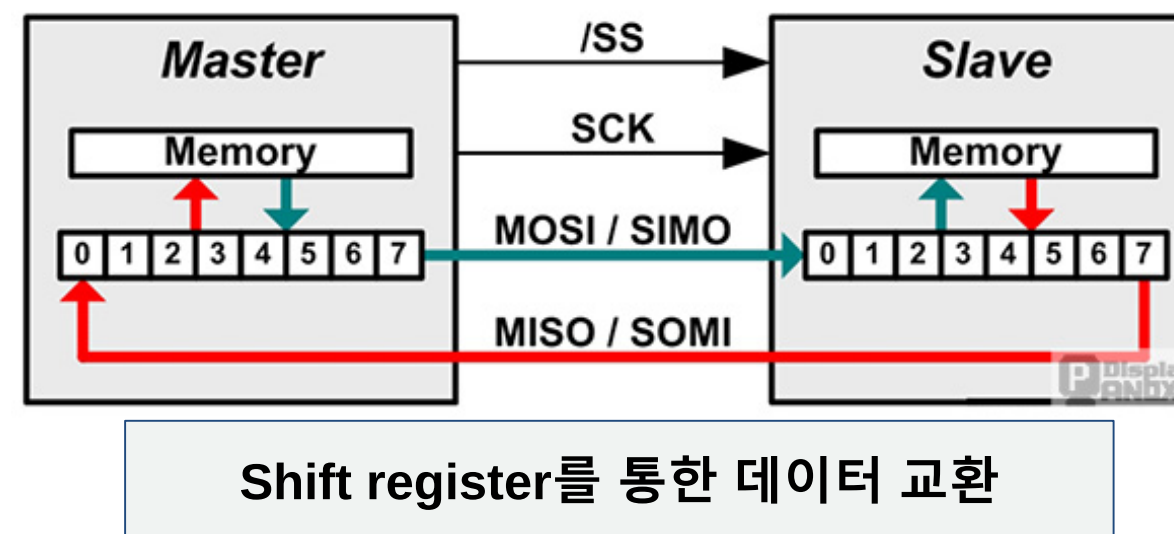
Master가 생성한 클럭을 기준으로 데이터를 동시에 송수신하는
고속 Full Duplex 통신

01

SPI 통신 Protocol



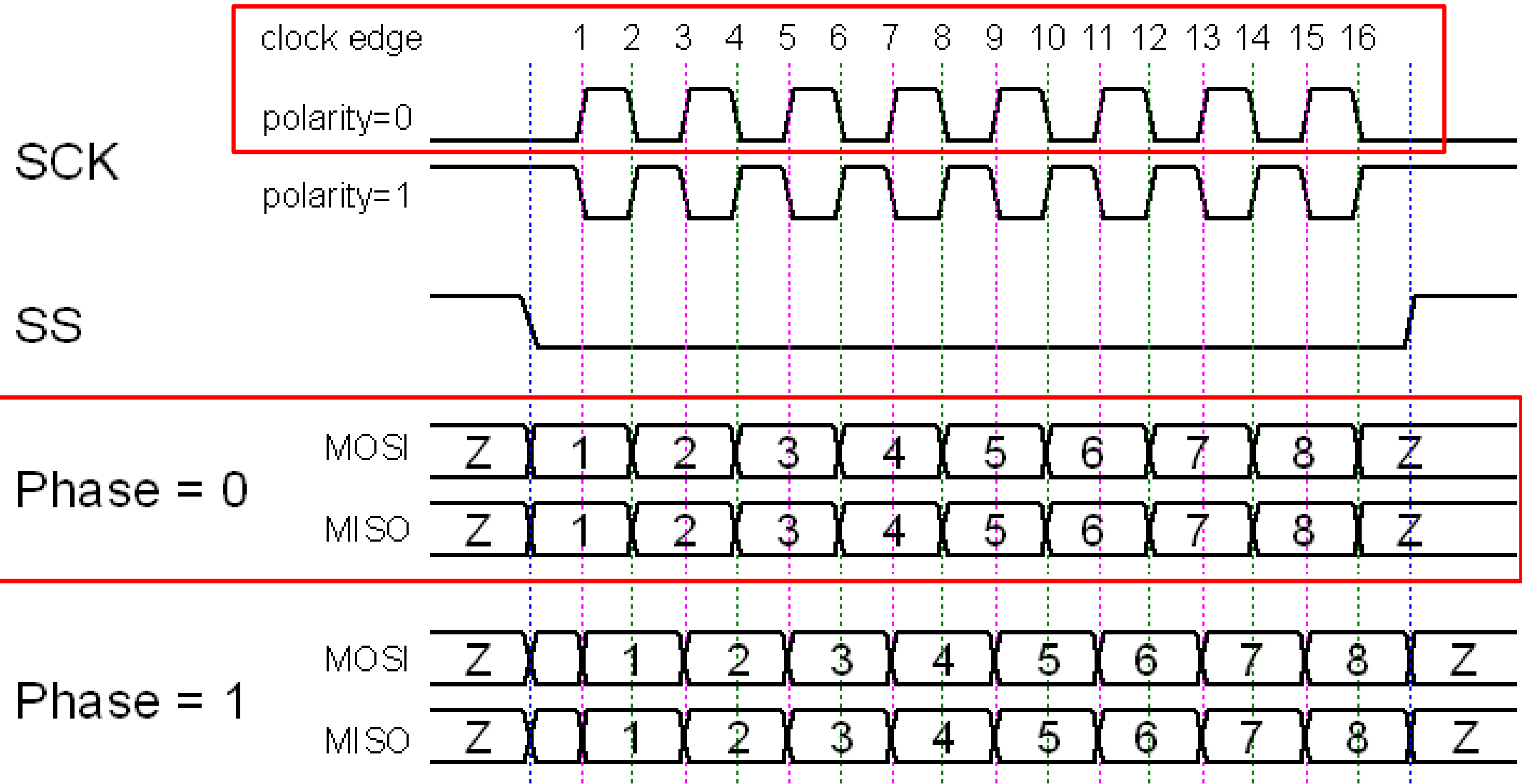
- **SCLK** : Master가 생성하는 clock
- **MOSI** : Master → Slave 데이터
- **MISO** : Slave → Master 데이터
- **CS_n** : Slave 선택 신호



SPI 특징

- 4-wire 버스
 - SCLK, MOSI, MISO, CS_N
- 동기식 통신
 - Master가 SCLK 생성
- Full - Duplex
 - 동시 송수신 가능
- 단일 Master, 다수 Slave
 - cs_n 개별
- CPOL/CPHA
 - 4가지 동작 모드

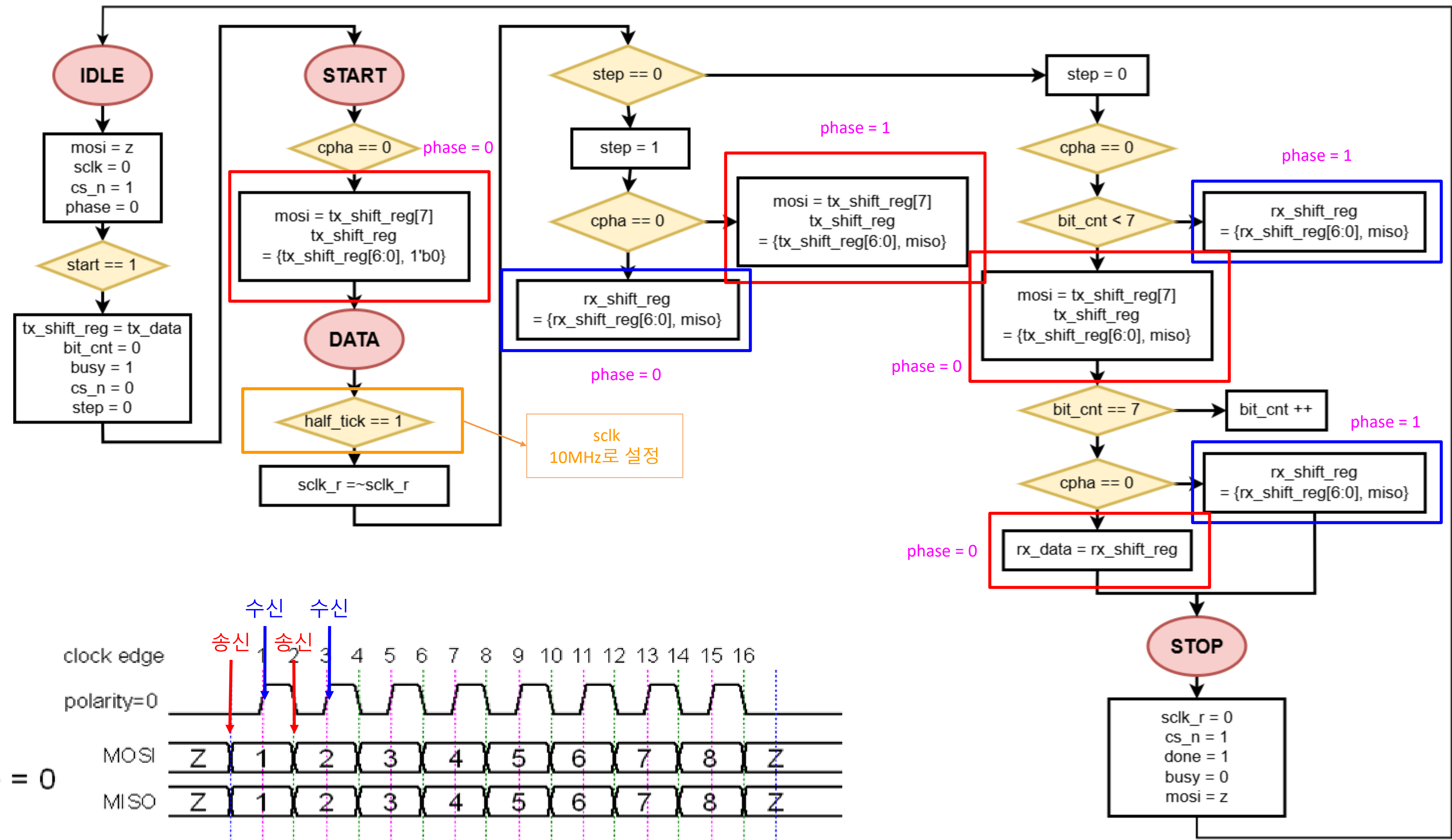
SPI 통신 Protocol



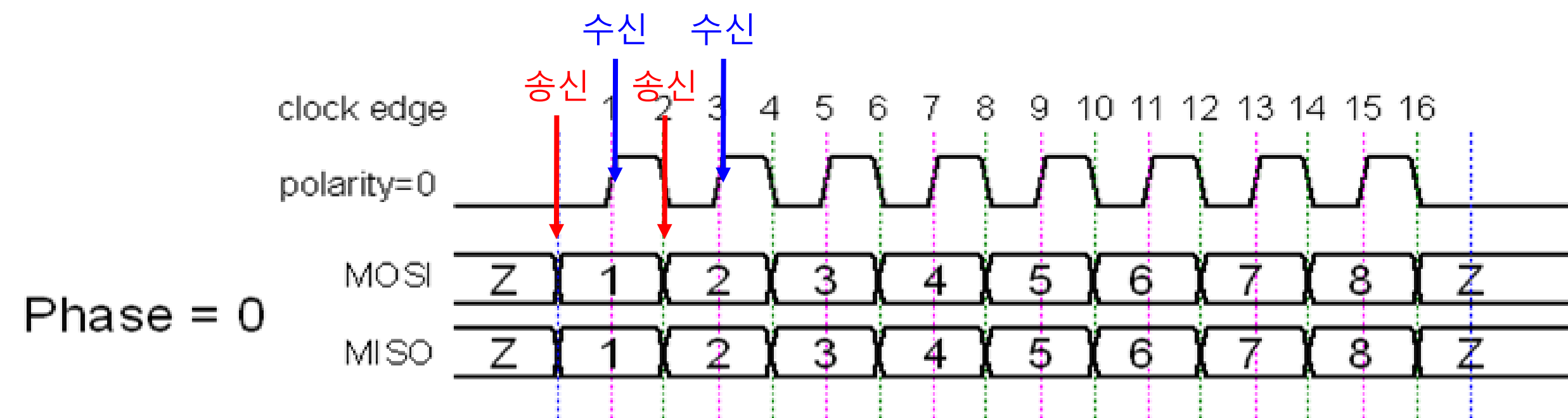
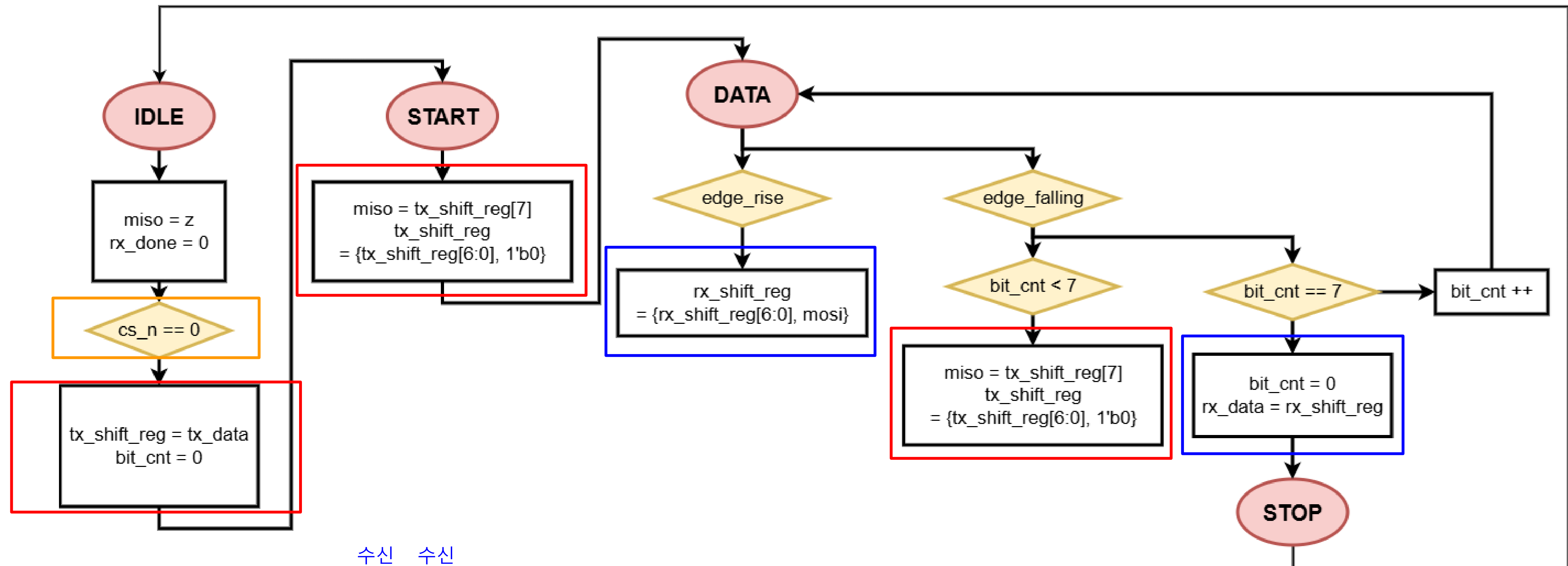
Mode	CPOL (클럭 극성)	CPHA (클럭 위상)	데이터 샘플링 시점
Mode 0	0	0	Rising Edge
Mode 1	0	1	Falling Edge
Mode 2	1	0	Falling Edge
Mode 3	1	1	Rising Edge

Protocol 스펙상 4가지 Mode가 존재
Mode 0만 지원하는 Slave가 가장 많이 사용됨

SPI Master ASM



SPI Slave ASM



SPI UVM 시나리오

Transaction 흐름		
01	Sequence	clk_div, m_tx_data, s_tx_data 랜덤 생성
02	Driver	start = 1 인가 -> m_done 대기 신호 Dut로 Drive
03	DUT	cs_n = 0 8bit mosi/miso 전송
04	Monitor	cs_n 하강 감지 sclk Low -> High 마다 Sampling

Monitor 타이밍	
sclk == 1 -> wait(sclk == 0) wait(sclk == 1) @(mon_cb)	
Scoreboard 비교 기준	
MOSI	exp : m_tx_data act : MOSI 수집 값
MISO	exp : s_tx_data act : MISO 수집 값

Coverage 항목
cp_clk_div 10MHz, 1MHz
cp_mosi / cp_miso 0x00~0xFF 전 범위
cp_tx_data 0x00~0xFF 전 범위

SPI UVM Coverage

```
==== Coverage Summary ====
Overall : 100.0%
MOSI data : 100.0%
MISO data : 100.0%
==== Coverage Summary ====

==== Coverage Summary ====
Overall : 100.0%
clk_div : 100.0%
m_tx_data : 100.0%
s_tx_data : 100.0%
==== Coverage Summary ====

***** SCB report *****
Result : ** PASS **
Total Transactions : 2560
Total Errors : 0
*****
```

8bit Data로 256
충분히 Test하기 위해 2560번 진행

MISO

Status	Bin Name	Type	At Least	Size	Hit Count
✓	data_all_ones	User	1	1	9
✓	data_all_zero	User	1	1	10
✓	data_high	User	1	1	658
✓	data_low	User	1	1	642
✓	data_mid_high	User	1	1	585
✓	data_mid_low	User	1	1	675
✓	data_toggle_...	User	1	1	13
✓	data_toggle_...	User	1	1	10

clk_div

Status	Bin Name	Type	At Least	Size	Hit Count
✓	clk_10M	User	1	1	1225
✓	clk_1M	User	1	1	1335

m_tx_data

Status	Bin Name	Type	At Least	Size	Hit Count
✓	high_half	User	1	1	1283
✓	low_half	User	1	1	1277

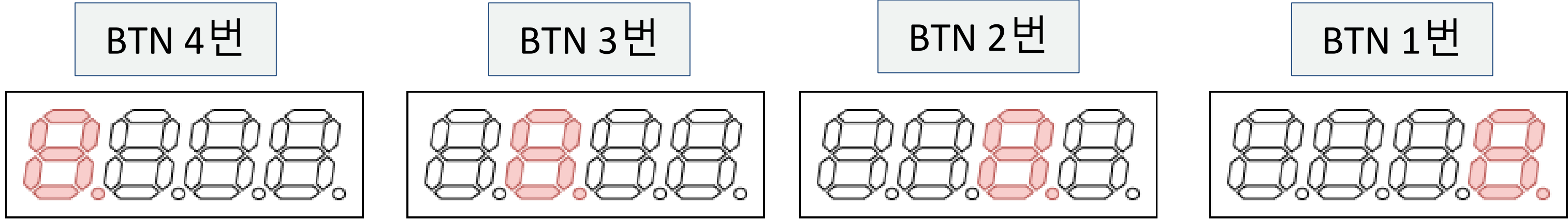
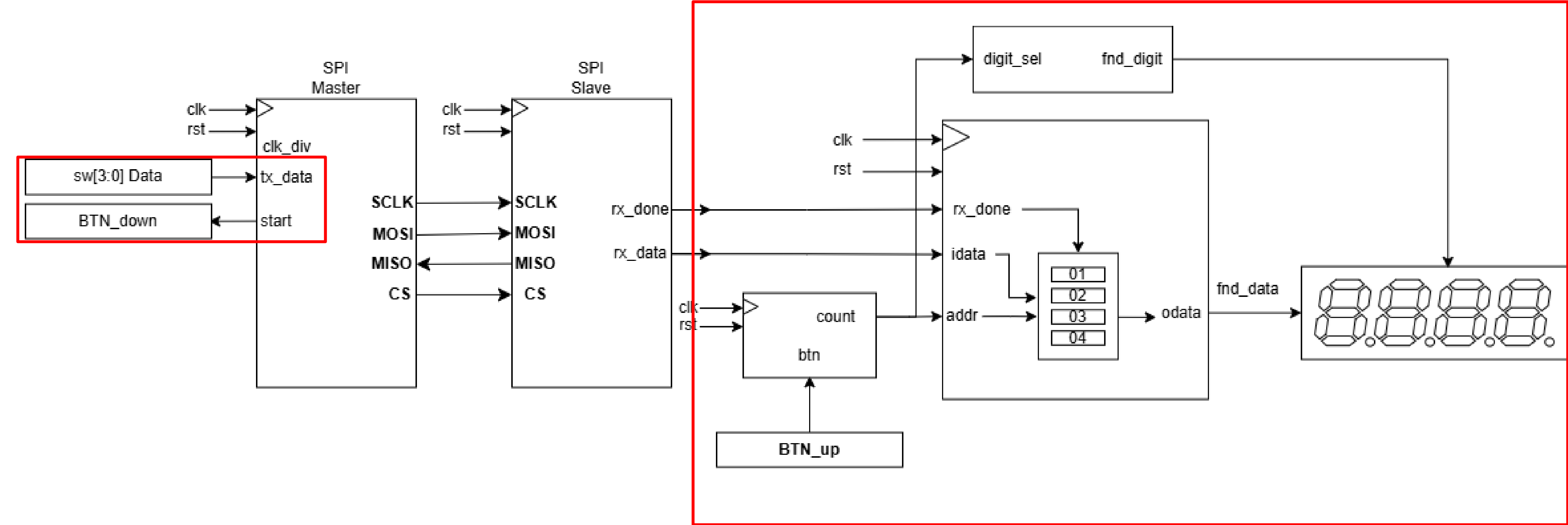
MOSI

Status	Bin Name	Type	At Least	Size	Hit Count
✓	data_all_ones	User	1	1	10
✓	data_all_zero	User	1	1	10
✓	data_high	User	1	1	621
✓	data_low	User	1	1	619
✓	data_mid_high	User	1	1	662
✓	data_mid_low	User	1	1	658
✓	data_toggle_...	User	1	1	8
✓	data_toggle_...	User	1	1	8

m_tx_data

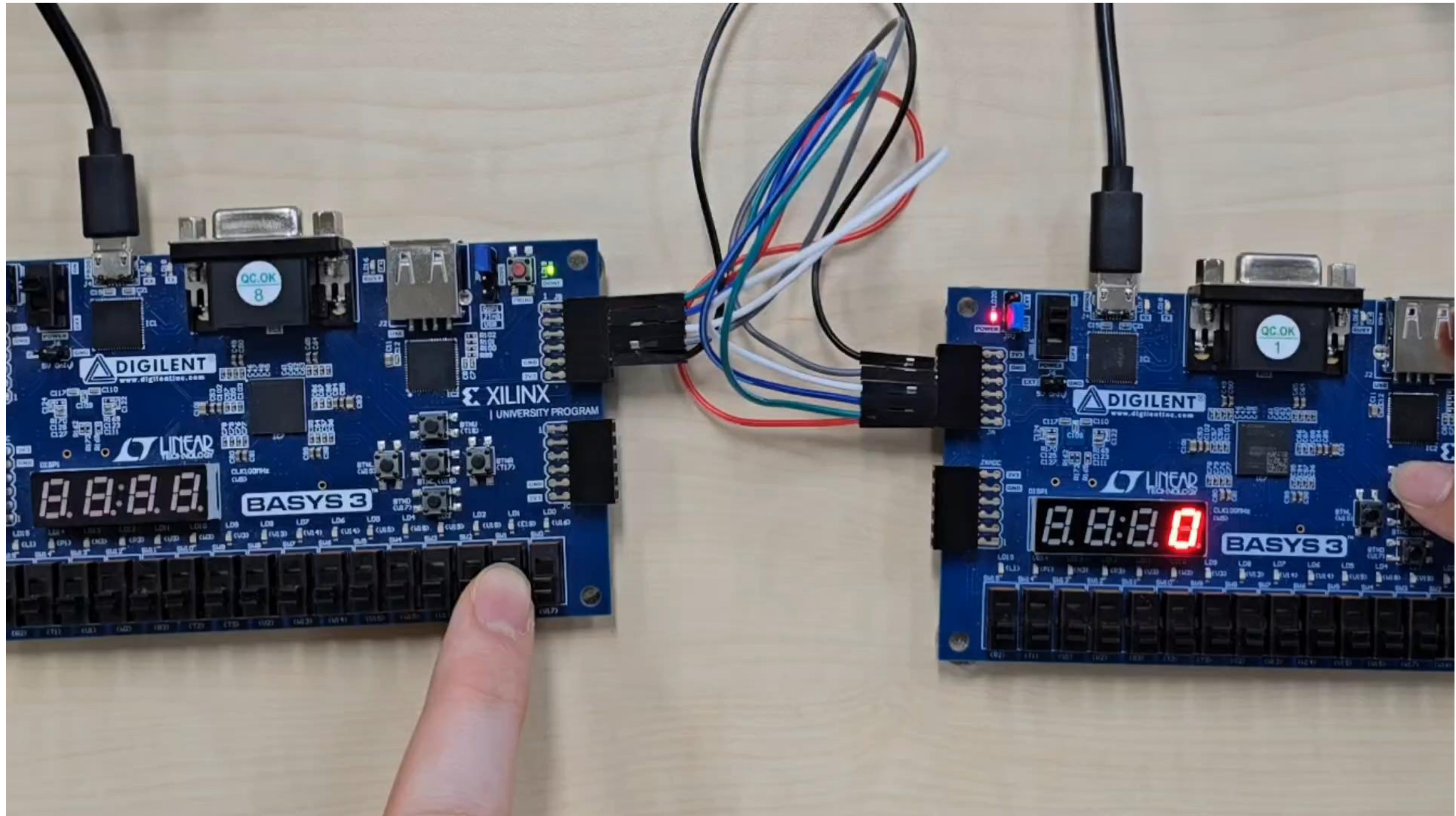
Status	Bin Name	Type	At Least	Size	Hit Count
✓	high_half	User	1	1	1243
✓	low_half	User	1	1	1317

Btn + sw + SPI + FND Block Diagram



rx_done == 1 일 때,
mem[addr] <= idata
btn counter가 주소 선택 역할

SPI 동작영상



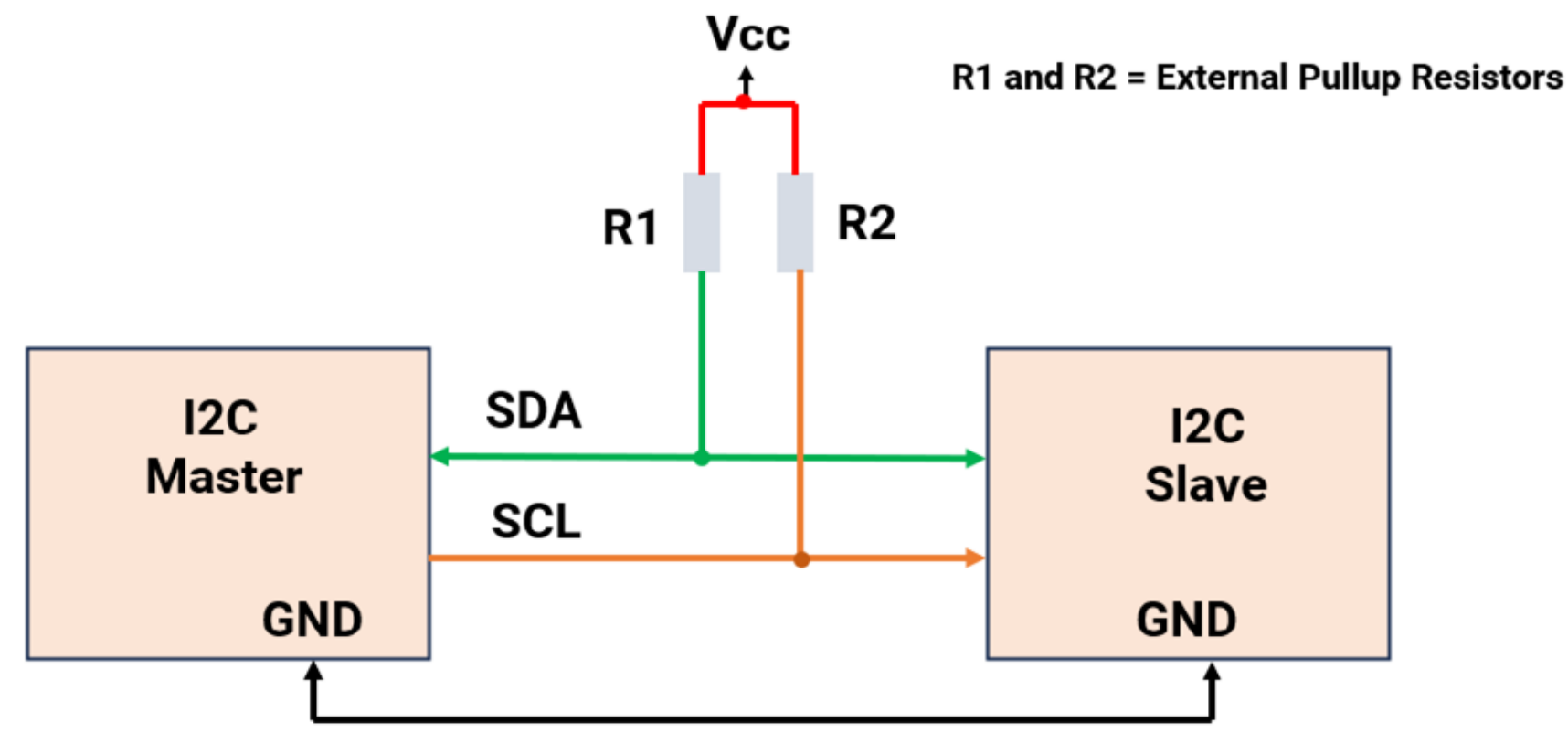
I2C



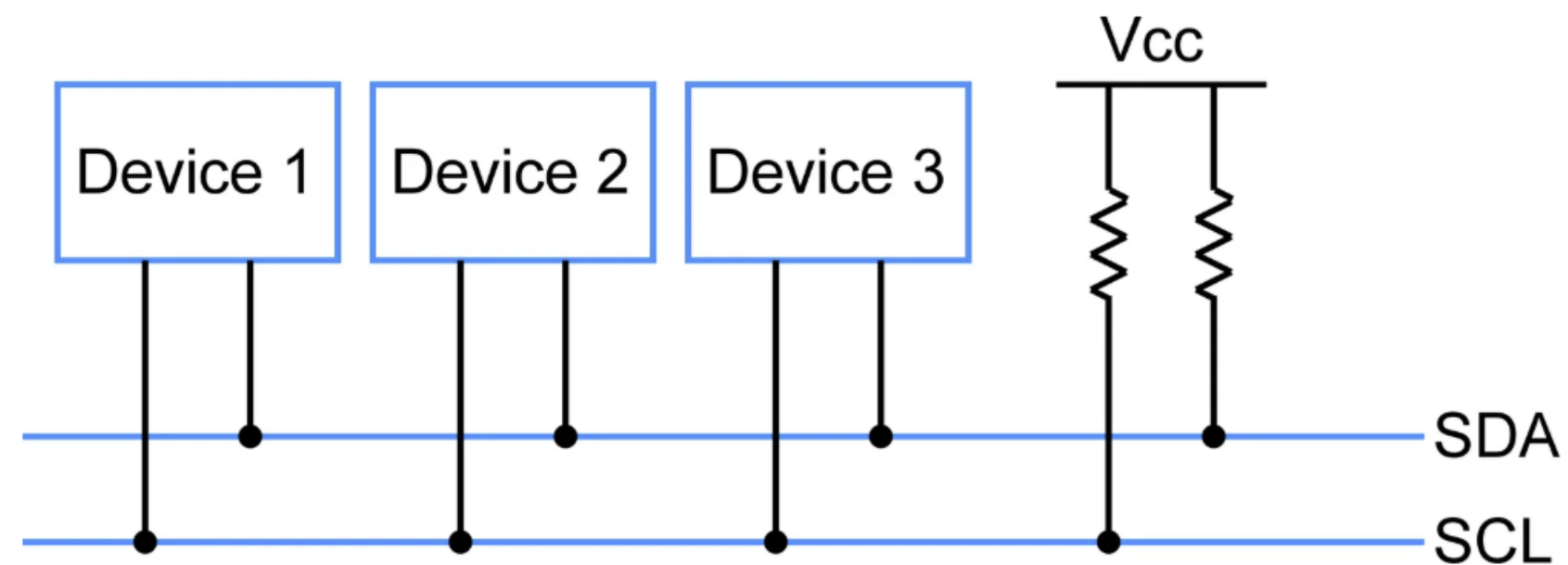
두 개의 선(SCL, SDA)을 사용하여 여러 디바이스가 주소 기반으로 통신하는
Half Duplex 방식

02

I2C 통신 Protocol



I2C Communication



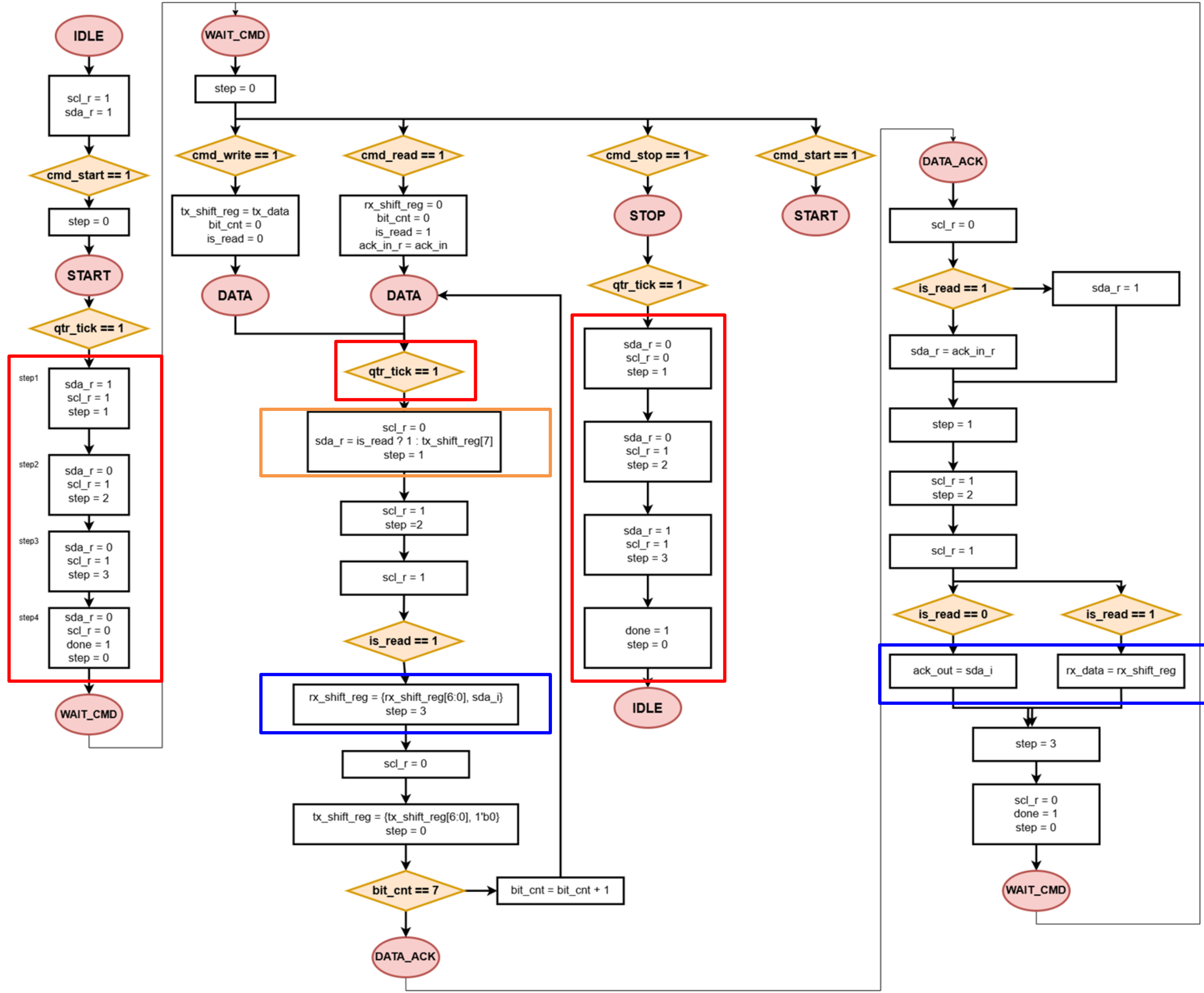
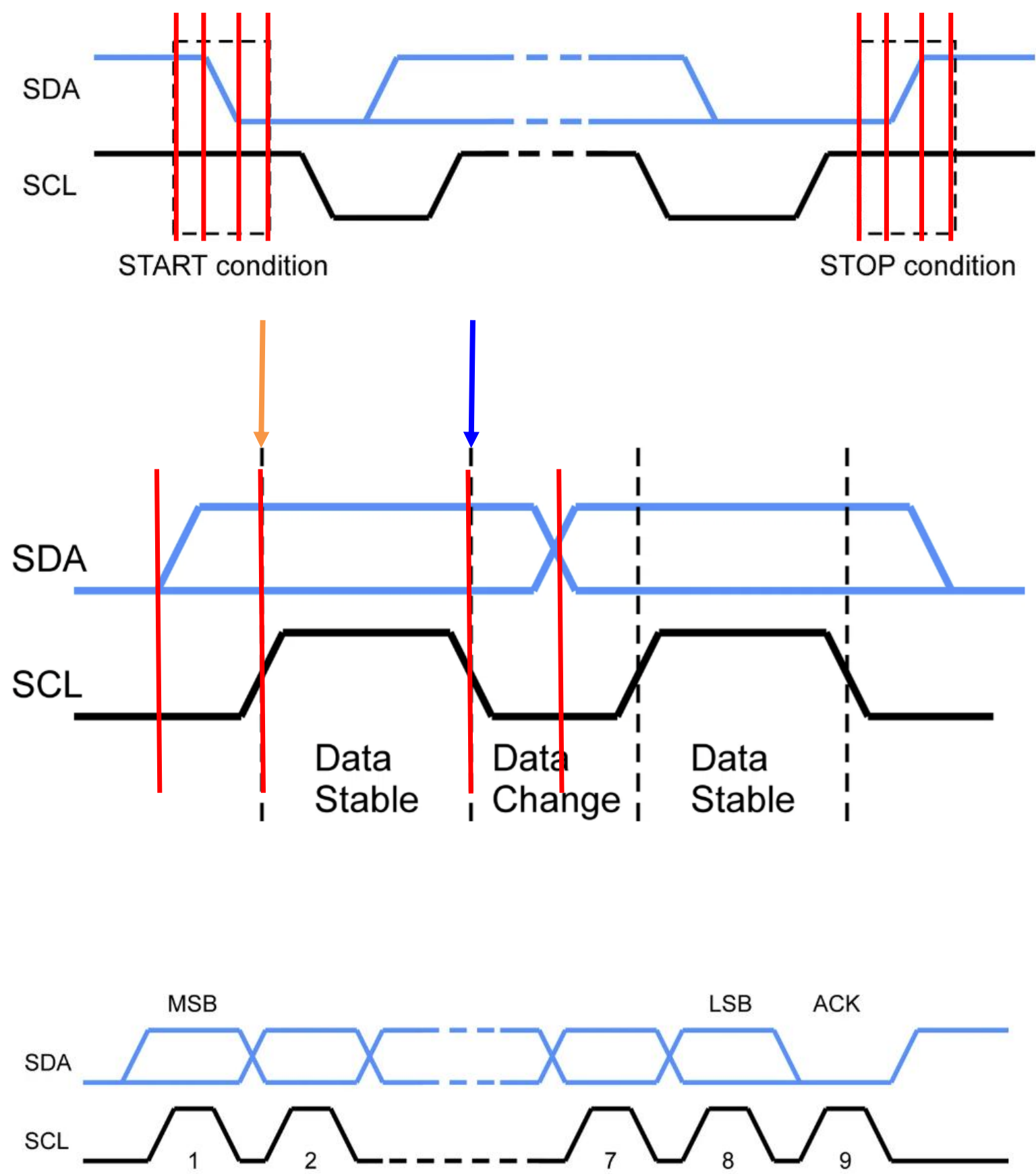
Transaction 흐름

- **START**
 - SCL = 1일 때, SDA fall
- **ADDR**
 - 7bit 주소 + R/W bit 전송
- **ACK**
 - Slave가 SDA = 0으로 응답
- **DATA**
 - 8bit Data 전송
- **STOP**
 - SCL = 1일 때, SDA rise

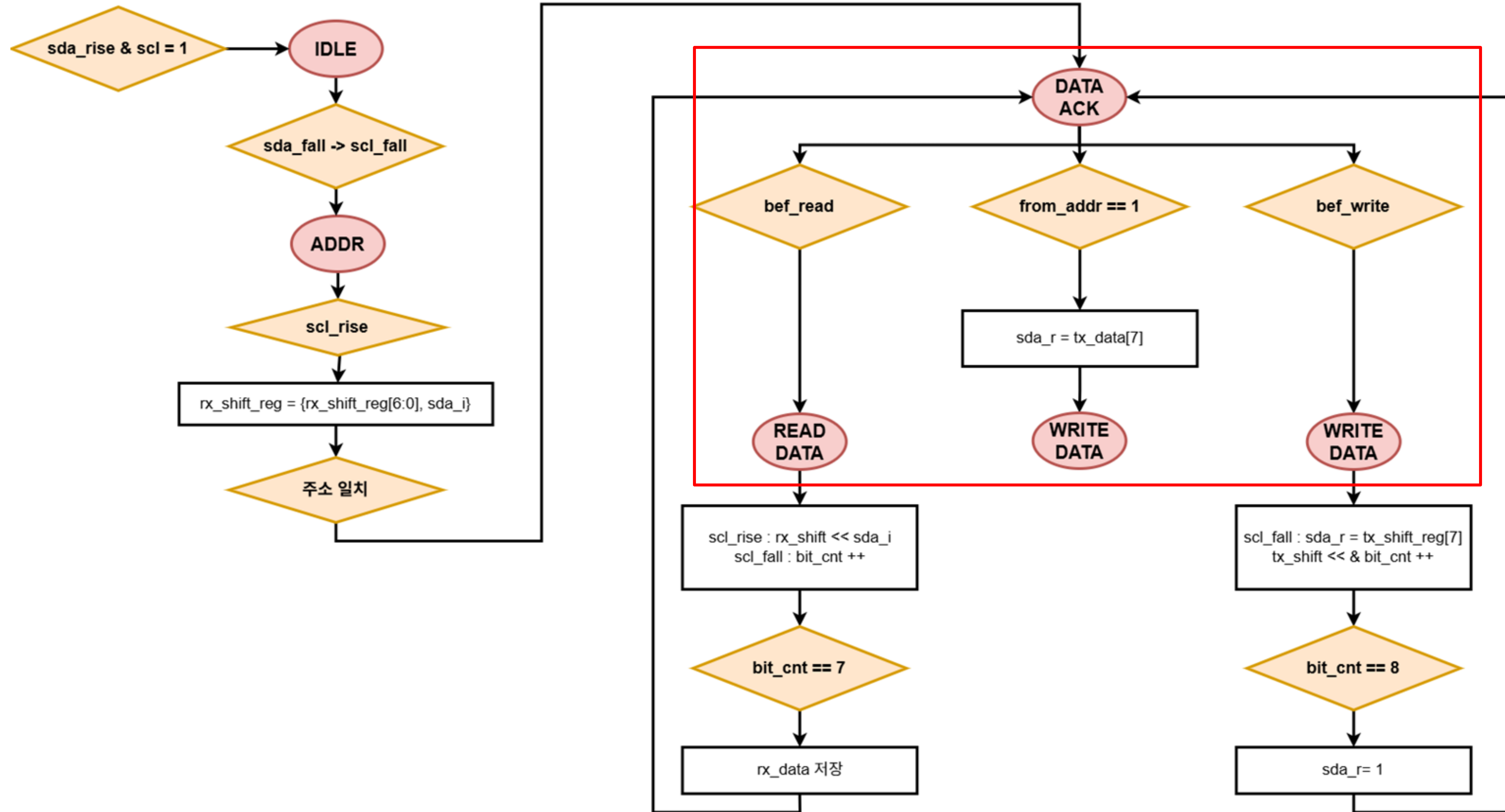
I2C 특징

- **2-wire 버스**
 - SCL, SDA (Open-Drain)
- **동기식 통신**
 - SCL 공유
- **Half - Duplex**
 - SDA 양방향 공유
- **주소 기반 통신**
 - 7bit/10bit Slave 주소
- **ACK/NACK**
 - Byte 마다 수신 확인

I2C ASM Master



I2C ASM Slave



I2C Simulation



I2C UVM 시나리오

Transaction 흐름		
01	Sequence	Command 신호 순서 생성 s_tx_data , m_tx_data 랜덤 생성
02	Driver	Dut에 신호 인가 m_done 대기 -> rx_data 읽기
03	Monitor	m_done 타이밍에 샘플링 ap.write
04	Scoreboard	drv_imp -> exp_queue push mon_imp -> exp_queue pop 비교하기

Monitor 타이밍	
@(mon_cb) cmd_write / cmd_read 감지 시 m_tx, s_tx 캡 쳐 m_done == 1 시점에 m_rx, s_rx 샘플링 주소 (0x24/0x25) 제외한 데이터만 ap.write is_write / is_read 플래그 초기화 (중복 방지)	
Scoreboard 비교 기준	
WRITE	exp : exp_item.m_tx_data act : item.s_rx_data
READ	exp : exp_item.m_tx_data act : item.m_rx_data

Coverage 항목	
cp_m_tx_data 0x00~0xFF	cp_m_tx_data 0x00~0xFF
cp_s_tx_data 0x00~0xFF	cp_s_tx_data 0x00~0xFF
cp_rw Write / Read	cp_rw Write / Read
cx_m_tx_rw m_tx x rw 교차	cx_m_rx_rw m_rx x rw 교차
cx_s_tx_rw s_tx x rw 교차	cx_s_rx_rw s_rx x rw 교차

I2C UVM Coverage

```
] ===== Monitor Coverage Summa
]
] Overall      : 100.0%
] m_rx_data   : 100.0%
] s_rx_data   : 100.0%
] rw          : 100.0%
] s_rx(W) cx  : 100.0%
] m_rx(R) cx  : 100.0%
]
] =====
]
] ===== Driver Coverage Summary
]
] Overall      : 100.0%
] m_tx_data   : 100.0%
] s_tx_data   : 100.0%
] rw          : 100.0%
```

8bit Data로 256
충분히 Test하기 위해 2560번 진행

m_tx_data 전체 범위 0x00~0xFF						
Status	Bin Name	Type	At Least	Size	Hit Count	
✓	full	User	1	1	3825	
✓	high	User	1	1	3933	
✓	low	User	1	1	3880	
✓	mid	User	1	1	3871	

s_tx_data 전체 범위 0x00~0xFF						
Status	Bin Name	Type	At Least	Size	Hit Count	
✓	read	User	1	1	7740	
✓	write	User	1	1	7769	

어떤 데이터를 Read/Write 했는지 교차 검증						
Status	Bin Name	Type	At Least	Size	Hit Count	
✓	full	User	1	1	3843	
✓	high	User	1	1	3877	
✓	low	User	1	1	3868	
✓	mid	User	1	1	3921	

어떤 데이터를 Read/Write 했는지 교차 검증						
Status	cp_m_tx_data	cp_rw	Type	At Least	Size	Hit Count
✓	full	read	Auto	1	1	1906
✓	full	write	Auto	1	1	1919
✓	high	read	Auto	1	1	1931
✓	high	write	Auto	1	1	2002
✓	low	write	Auto	1	1	1911
✓	low	read	Auto	1	1	1969
✓	mid	read	Auto	1	1	1934
✓	mid	write	Auto	1	1	1937

어떤 데이터를 Read/Write 했는지 교차 검증						
Status	cp_s_tx_data	cp_rw	Type	At Least	Size	Hit Count
✓	full	write	Auto	1	1	1889
✓	full	read	Auto	1	1	1954
✓	high	read	Auto	1	1	1897
✓	high	write	Auto	1	1	1980
✓	low	read	Auto	1	1	1924
✓	low	write	Auto	1	1	1944
✓	mid	write	Auto	1	1	1956
✓	mid	read	Auto	1	1	1965

Master가 수신한 실제 데이터 (Read 시)						
Status	Bin Name	Type	At Least	Size	Hit Count	
✓	full	User	1	1	3825	
✓	high	User	1	1	3933	
✓	low	User	1	1	3880	
✓	mid	User	1	1	3871	

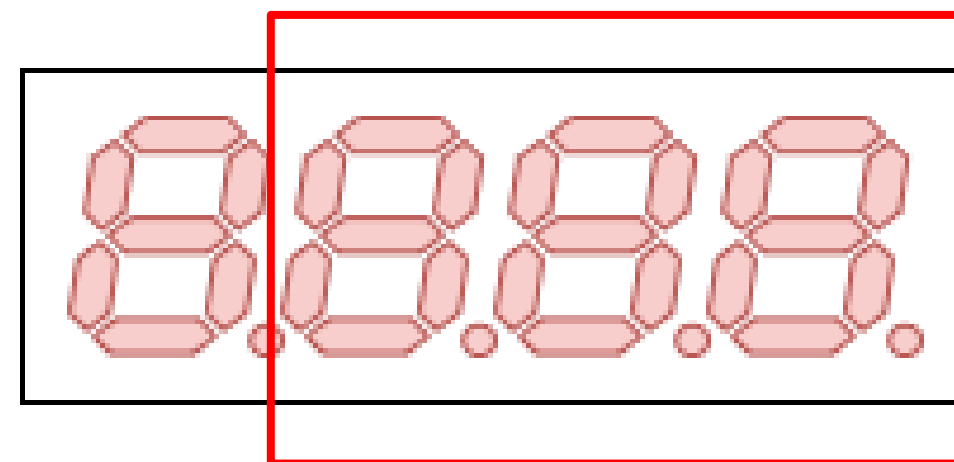
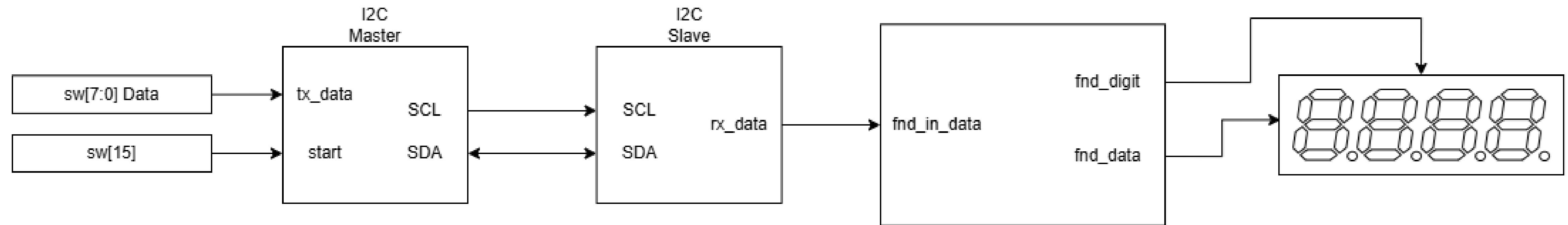
write / read 방향						
Status	Bin Name	Type	At Least	Size	Hit Count	
✓	read	User	1	1	7740	
✓	write	User	1	1	7769	

Slave가 수신한 실제 데이터 (Write 시)						
Status	Bin Name	Type	At Least	Size	Hit Count	
✓	full	User	1	1	3843	
✓	high	User	1	1	3877	
✓	low	User	1	1	3868	
✓	mid	User	1	1	3921	

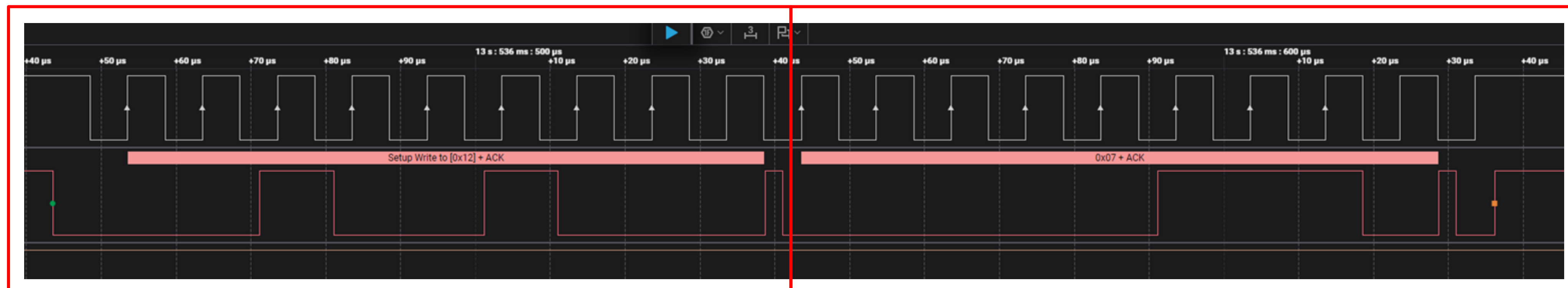
Slave가 모든 구간의 데이터를 잘 받았는지 (Write)						
Status	cp_m_tx_data	cp_rw	Type	At Least	Size	Hit Count
✓	full	read	Auto	1	1	1906
✓	full	write	Auto	1	1	1919
✓	high	read	Auto	1	1	1931
✓	high	write	Auto	1	1	2002
✓	low	write	Auto	1	1	1911
✓	low	read	Auto	1	1	1969
✓	mid	read	Auto	1	1	1934
✓	mid	write	Auto	1	1	1937

Master가 모든 구간의 데이터를 잘 받았는지 (Read)						
Status	cp_s_tx_data	cp_rw	Type	At Least	Size	Hit Count
✓	full	write	Auto	1	1	1889
✓	full	read	Auto	1	1	1954
✓	high	read	Auto	1	1	1897
✓	high	write	Auto	1	1	1980
✓	low	read	Auto	1	1	1924
✓	low	write	Auto	1	1	1944
✓	mid	write	Auto	1	1	1956
✓	mid	read	Auto	1	1	1965

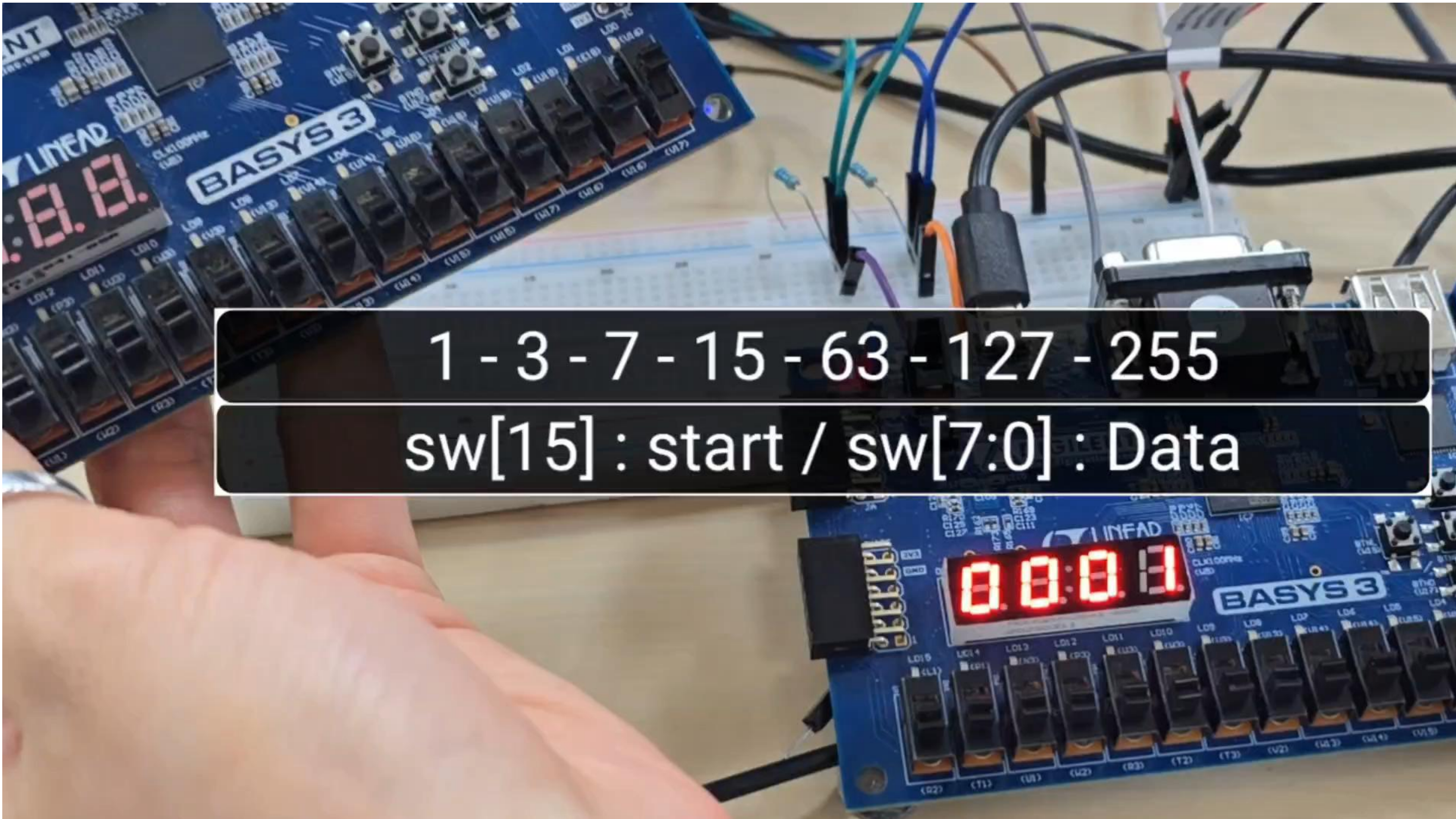
SW + I2C + FND Block Diagram



sw 값을 읽어서
0부터 255까지 나타낼 수 있음
주소는 고정해두고, sw[15]를 start로 사용함



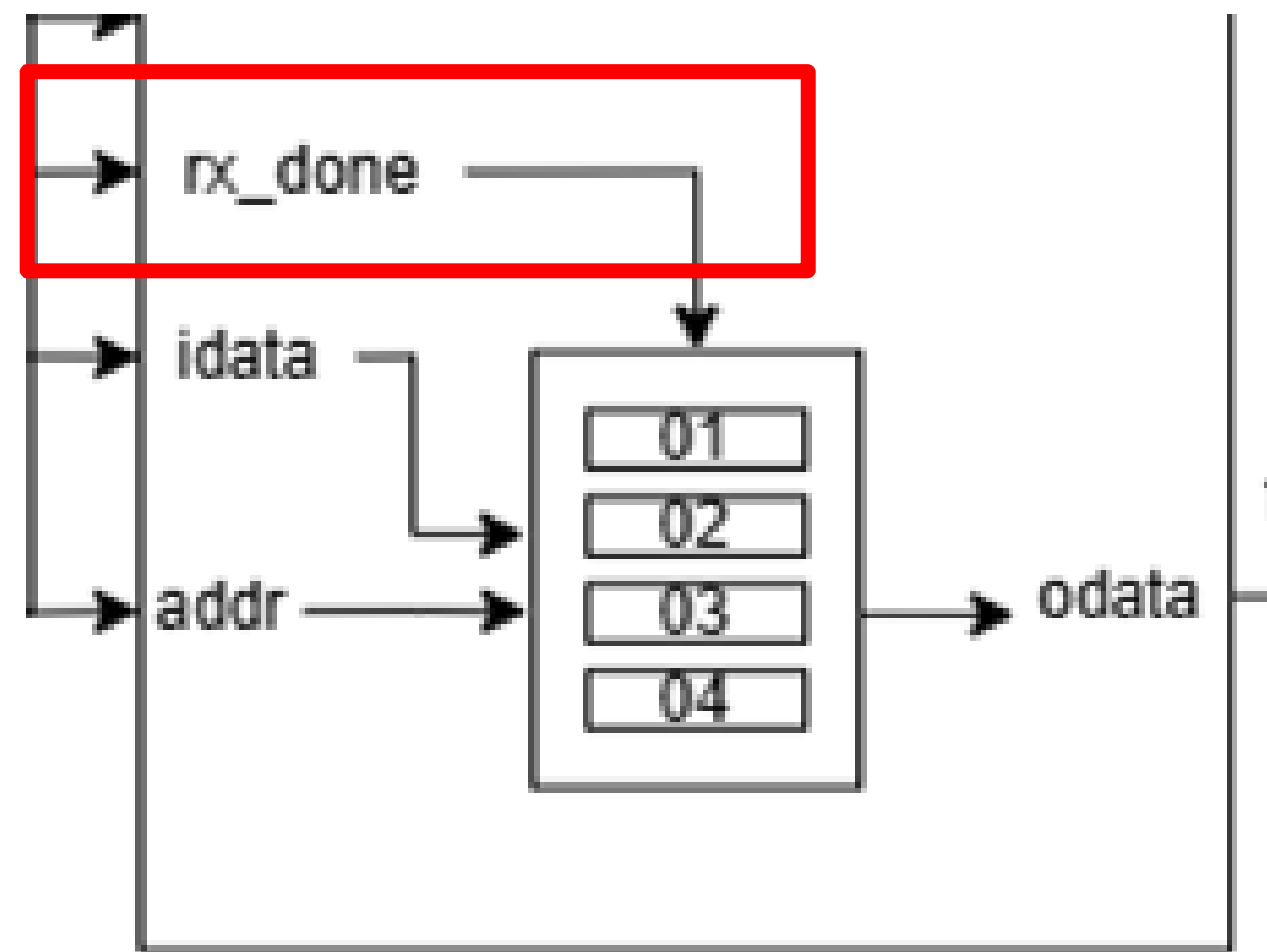
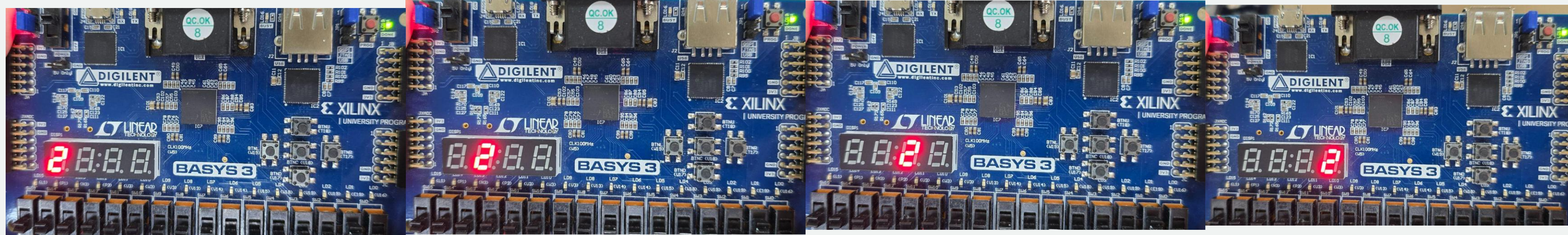
I2C 동작영상



Troubleshooting

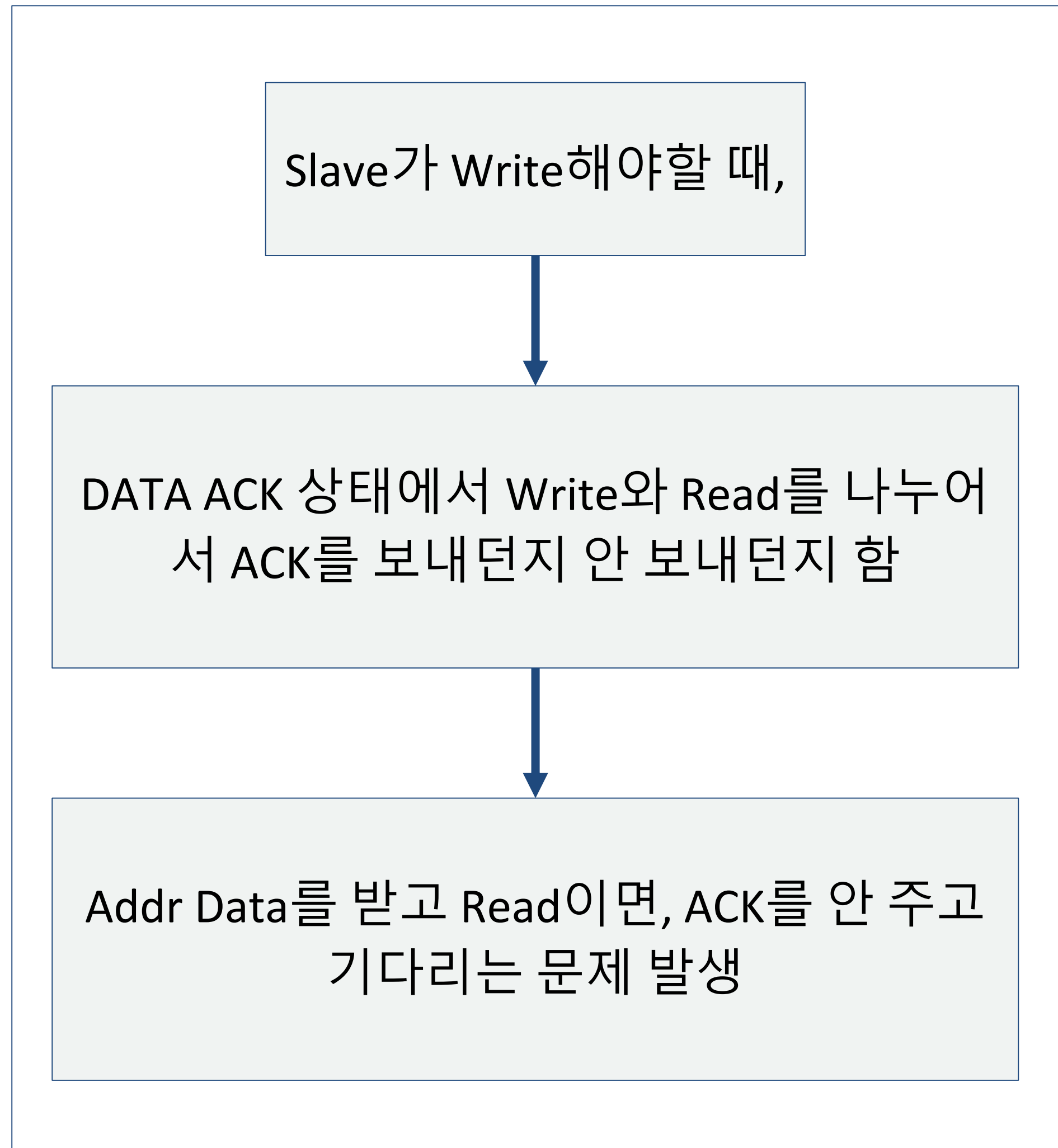
03

Troubleshooting : SPI FND

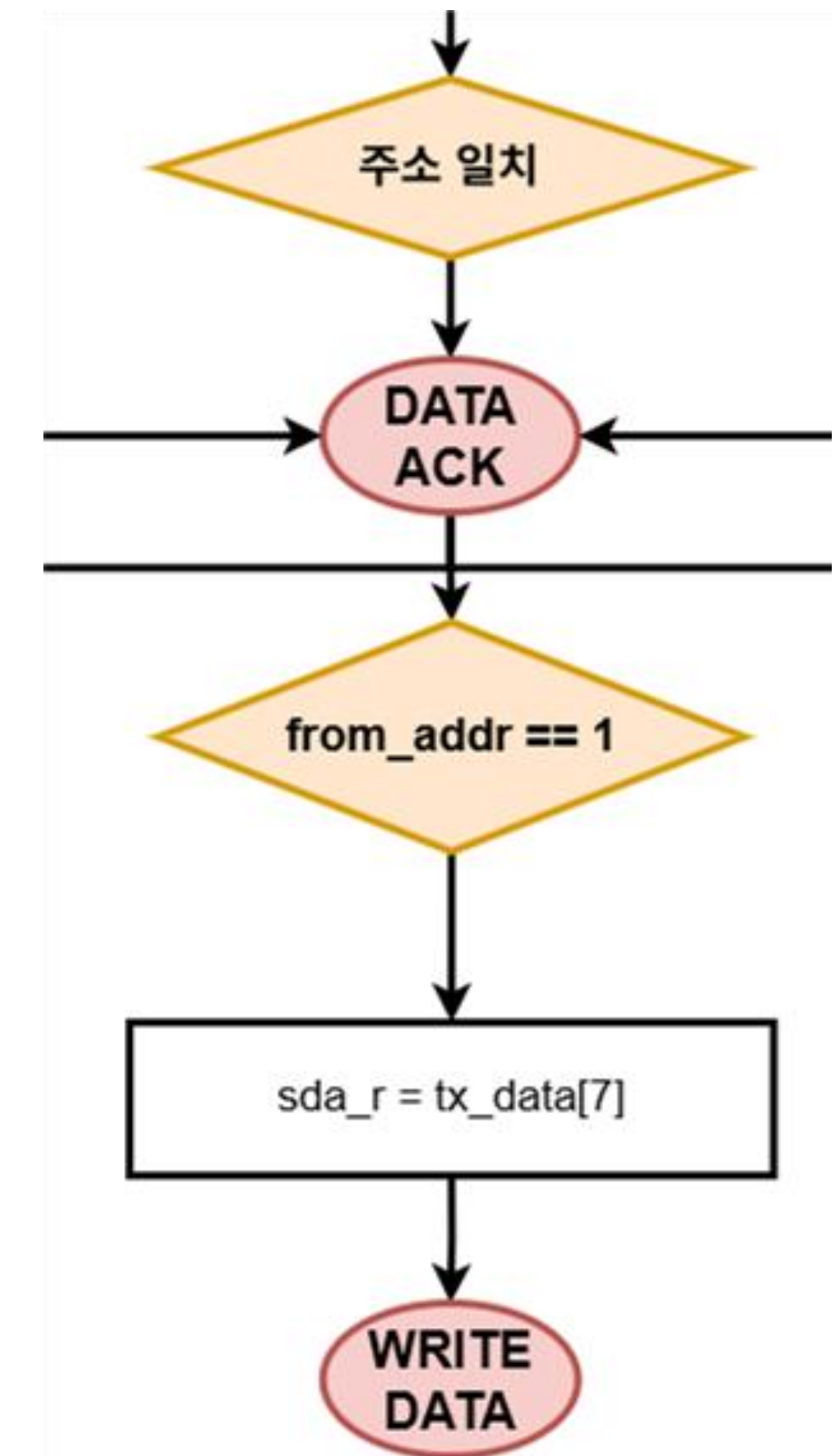


BTN count로 주소 지정을 원했지만,
매번, 새로운 값이 들어가게 됨
rx_done 신호 추가해서,
rx_done 일 때만, 값 저장

Troubleshooting : I2C Slave 설계



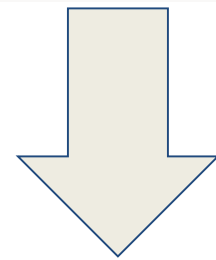
from_addr라는 기준 추가



Troubleshooting : SPI UVM

```
for (int i = 0; i < 8; i++) begin
    wait (spi_if.mon_cb.sclk == 1'b1);
    @(spi_if.mon_cb);
    shift_mosi = {shift_mosi[6:0], spi_if.mon_cb.mosi};

    if (i < 7) begin
        wait (spi_if.mon_cb.sclk == 1'b1);
        @(spi_if.mon_cb);
    end
end
```



```
for (int i = 0; i < 8; i++) begin
    // sclk이 HIGH면 먼저 LOW로 내려갈 때까지 대기
    if (spi_if.mon_cb.sclk == 1'b1) begin
        wait (spi_if.mon_cb.sclk == 1'b0); // LOW 확인
    end
    // LOW 확인 후 상승엿지 대기
    wait (spi_if.mon_cb.sclk == 1'b1);
    @(spi_if.mon_cb);
    shift_mosi = {shift_mosi[6:0], spi_if.mon_cb.mosi};
end
```

wait : 현재 값이 이미 조건을 만족하면 기다리지 않고 즉시 통과
HIGH 구간 중에 다시 wait(sclk==1)을 만나면 그냥 통과 → 오류
LOW를 먼저 확인하는 패턴으로 변경

감사합니다.